Software Design Document (SDD)

Module	GPIO Driver
Version	1.0.0
Team	Carlos Araiza · Carlos Cayetano · Kheara Kieley · Vania Leal
Date	26/04/2026
Institution	CETYS Universidad

---

1. Introduction

1.1. Purpose

This document presents the design and internal architecture of the GPIO (General Purpose Input/Output) driver module developed for the STM32F411RE microcontroller. The design is grounded in the requirements defined in the corresponding Software Requirements Specification (SRS) and covers the key data structures, hardware register interactions, control flow, and the full public API providing a complete technical reference for the implementation and validation of any software module that builds on GPIO functionality.

1.2. Scope

The GPIO driver is a bare-metal, low-level firmware component for the STM32F411RE embedded platform. It wraps the microcontroller's GPIO hardware registers behind a structured, validated API, through which higher-level modules can enable port clocks, configure individual pin modes, and perform read, write, and toggle operations on any pin within ports A through E. Pin numbers 0 through 15 are supported on each port. The driver depends exclusively on CMSIS device headers and performs all hardware access through direct register manipulation; no STM32 HAL or third-party libraries are involved.

1.3. Definitions and Acronyms

Term	Definition
GPIO	General Purpose Input/Output — individually configurable digital pins on a microcontroller, usable for input sensing or output driving.
STM32F411RE	ARM Cortex-M4 based microcontroller by STMicroelectronics. Target device for this driver, present on the Nucleo-F411RE development board.
RCC	Reset and Clock Control — STM32F4 peripheral responsible for gating the bus clock to each GPIO port via the AHB1ENR register.
AHB1	Advanced High-performance Bus 1 — the internal bus on which STM32F4 GPIO peripherals reside at fixed memory offsets.
MODER	Mode Register — holds a 2-bit field per pin selecting its function: input (00), output (01), alternate function (10), or analog (11).
ODR	Output Data Register — readable/writable; controls the logic level driven on each output-configured pin.
IDR	Input Data Register — read-only; always mirrors the actual voltage level present on each physical pin.
BSRR	Bit Set/Reset Register — write-only; bits [15:0] set pins HIGH atomically, bits [31:16] clear them LOW, each in a single write.
CMSIS	Cortex Microcontroller Software Interface Standard — provides vendor-supplied register-map structures so the driver avoids hard-coded memory addresses.
SWD	Serial Wire Debug — two-wire interface (PA13/PA14) used by the ST-Link on the Nucleo board for programming and debugging.
SRS	Software Requirements Specification — the document defining the behavioral requirements this design implements.
SDD	Software Design Document — this document.
API	Application Programming Interface — the set of public functions through which external modules interact with the driver.
FR	Functional Requirement — a requirement specifying a concrete behavior the system must exhibit.
NFR	Non-Functional Requirement — a requirement describing a system quality attribute such as performance or portability.

1.4. Requirements

All functional requirements for this module are defined in the Software Requirements Specification (SRS) for the GPIO driver. The table below traces each SRS requirement to the public API function that fulfills it.

Req. ID	Function	Requirement Statement
FR-1	gpio_init()	The driver shall reset all GPIO ports to a known default state upon startup.
FR-2	gpio_initPort()	The driver shall enable the peripheral clock for any individually specified GPIO port.
FR-3	gpio_setPinMode()	The driver shall allow the operating mode of any individual pin to be configured.
FR-4	gpio_setPin()	The driver shall drive a specified output pin to logic HIGH.
FR-5	gpio_clearPin()	The driver shall drive a specified output pin to logic LOW.
FR-6	gpio_togglePin()	The driver shall invert the current output state of a specified pin.
FR-7	gpio_readPin()	The driver shall sample and return the current digital state of any specified pin.
FR-8	gpio_setAlternateFunction()	The driver shall configure the alternate function mapping for a specified pin.

The SRS additionally establishes NFR-1 (no external library dependencies), NFR-2 (input validation with structured error codes on all public functions), and NFR-3 (deterministic execution time suitable for real-time embedded use). Hardware references are the STM32F411xC/E reference manual RM0383 and the device datasheet.

2. System Architecture

2.1. High-Level Design

The GPIO driver adopts a layered software architecture that enforces a clean boundary between application-level logic and low-level hardware access. The three layers that make up the system are described below:

Layer	Description
Application Layer	Higher-level software modules that call the GPIO driver API to interact with hardware. No application code accesses GPIO or RCC registers directly.
GPIO Driver Layer	Implements the public API declared in gpio_driver.h. Validates all input parameters through the internal is_valid() helper, enables port clocks via RCC→AHB1ENR, and performs direct register operations on MODER, BSRR, ODR, IDR, and AFR using the CMSIS GPIO_TypeDef and RCC_TypeDef structures.

Hardware Layer	Physical STM32F411RE GPIO peripheral registers located at fixed AHB1 bus addresses, accessed through CMSIS typed pointer macros (GPIOA … GPIOE, RCC).

2.2. Module Interfaces

The GPIO driver makes its complete functionality available through the single public header file gpio_driver.h. The module's interface characteristics are summarized below:

- **Public API:** All externally callable functions are declared in gpio_driver.h. These are gpio_init, gpio_initPort, gpio_setPinMode, gpio_setPin, gpio_clearPin, gpio_togglePin, gpio_readPin, and gpio_setAlternateFunction.
- **Internal implementation:** All function bodies and the private helper is_valid() are defined in gpio_driver.c and are not accessible to external modules.
- **Dependencies:** The driver depends on the CMSIS STM32F4xx device header (stm32f4xx.h) for the GPIO_TypeDef and RCC_TypeDef register-map structures, and on stdint.h for fixed-width integer types. No HAL or external libraries are used; all hardware access is through direct register manipulation.
- **Hardware dependencies:** STM32F411RE GPIO peripheral registers at their AHB1 fixed base addresses, and the RCC AHB1ENR register for clock gating.

3. Detailed Design

3.1. Class Diagram

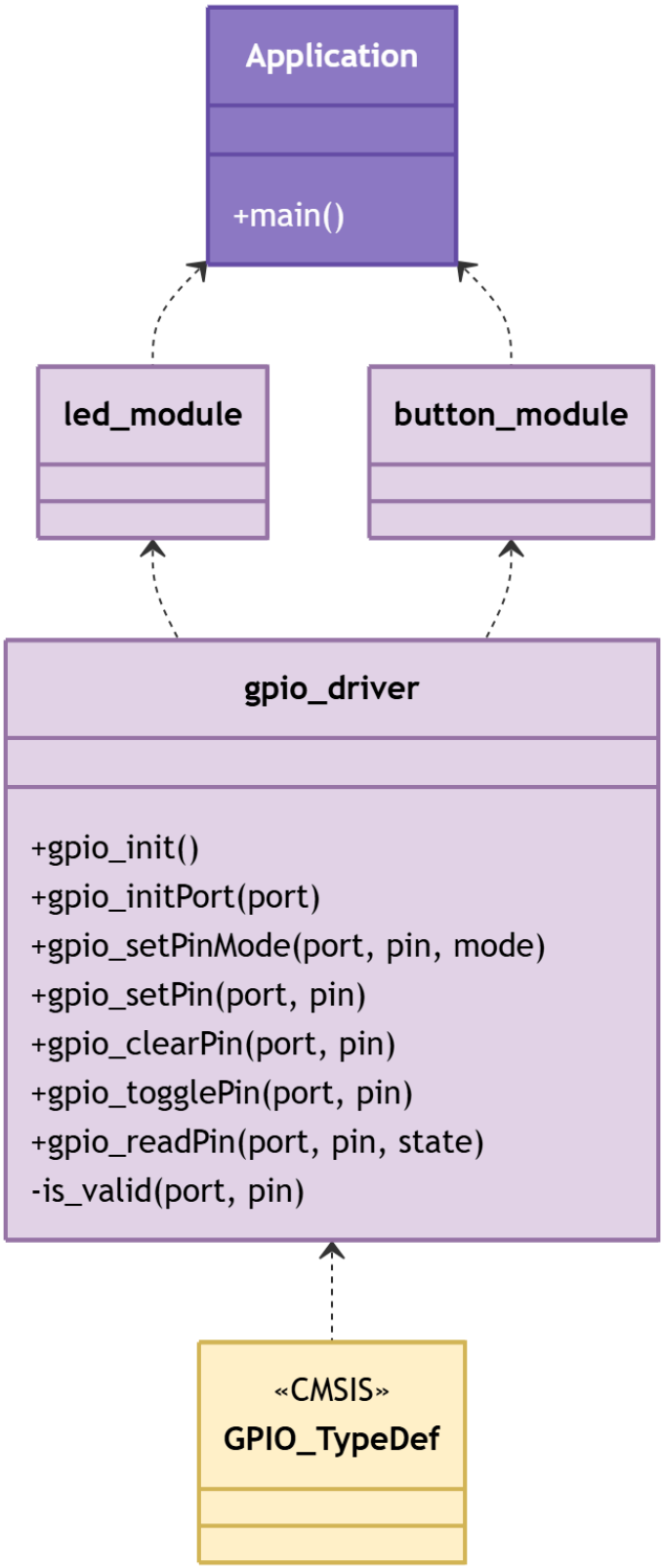

Figure 1. Class Diagram - Driver

3.2. Module Behavior (Sequence Diagrams, State Machines, etc.)

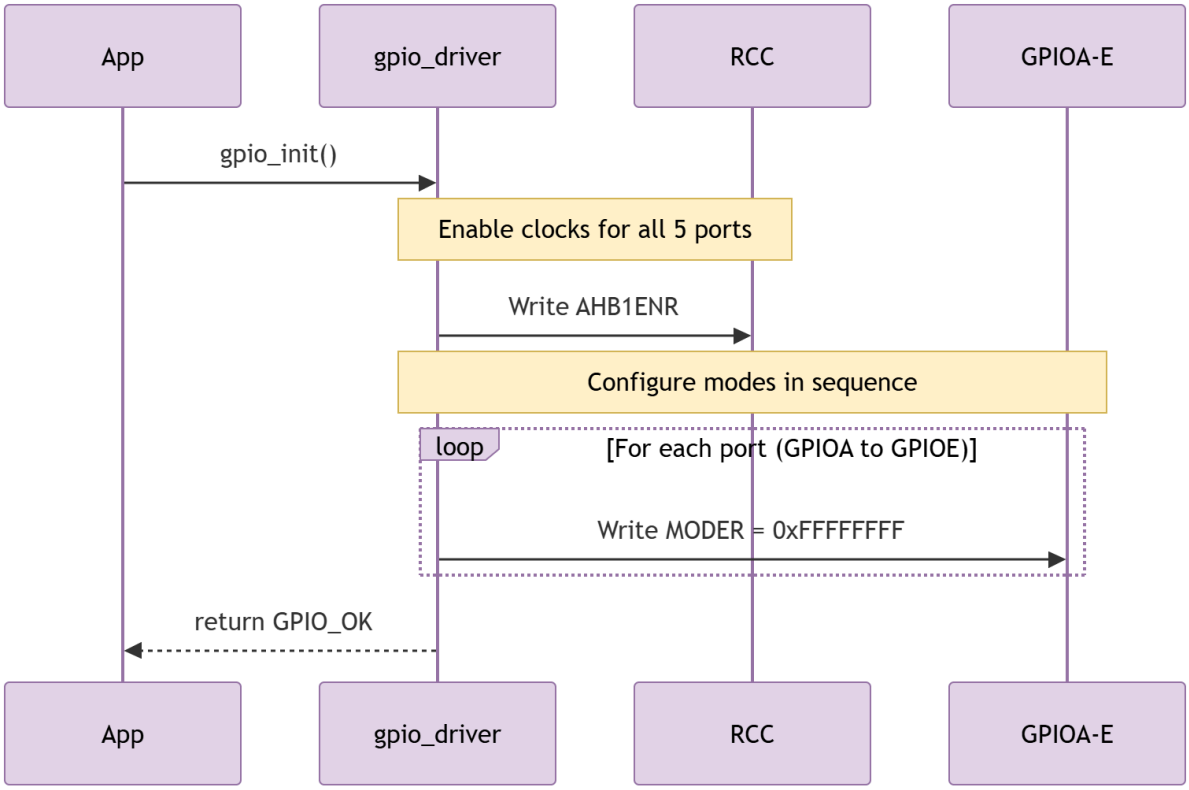

Figure 2. Sequence Diagram - Driver Init

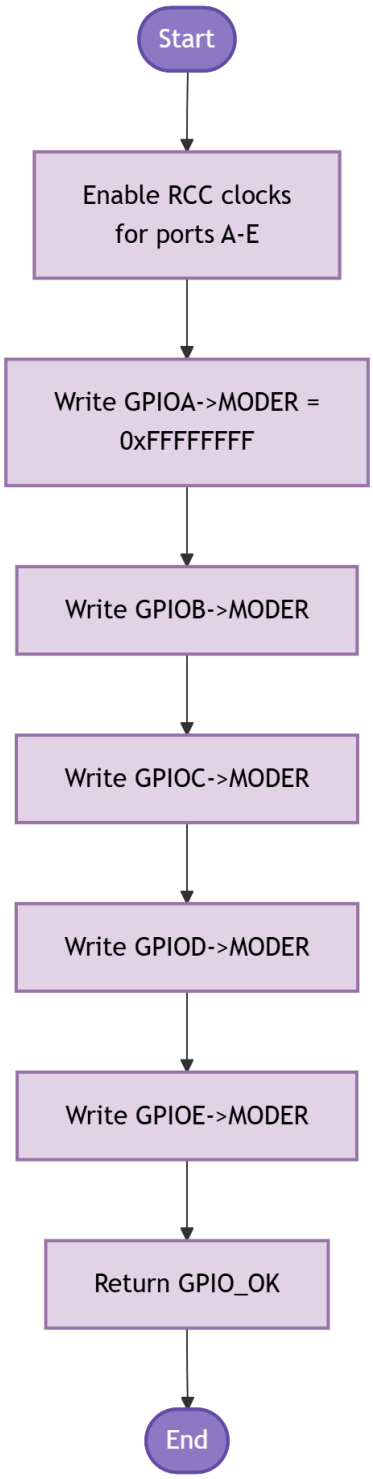

Figure 3. Activity Diagram - Driver Init

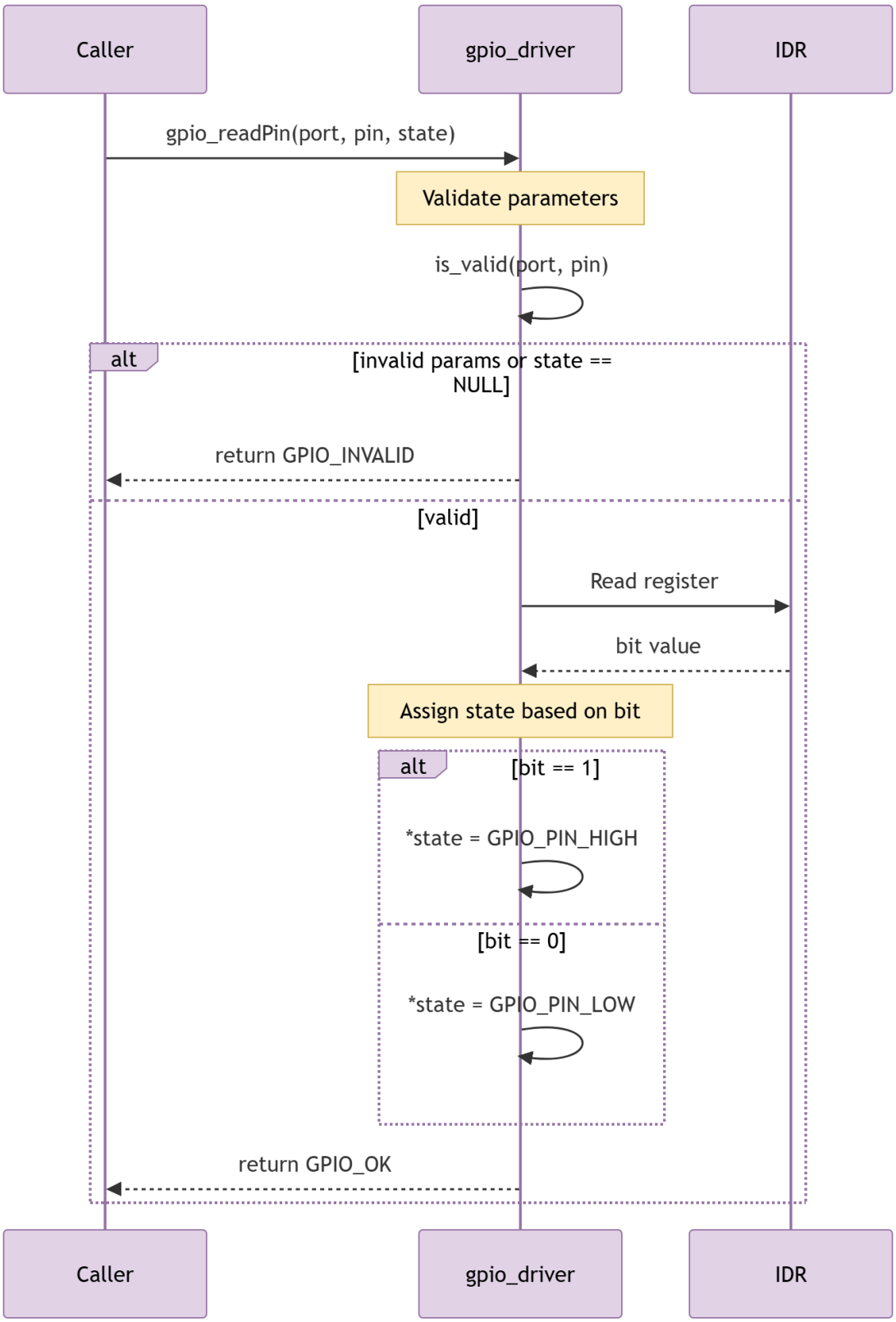

Figure 4. Sequence Diagram - Driver Action with Conditional

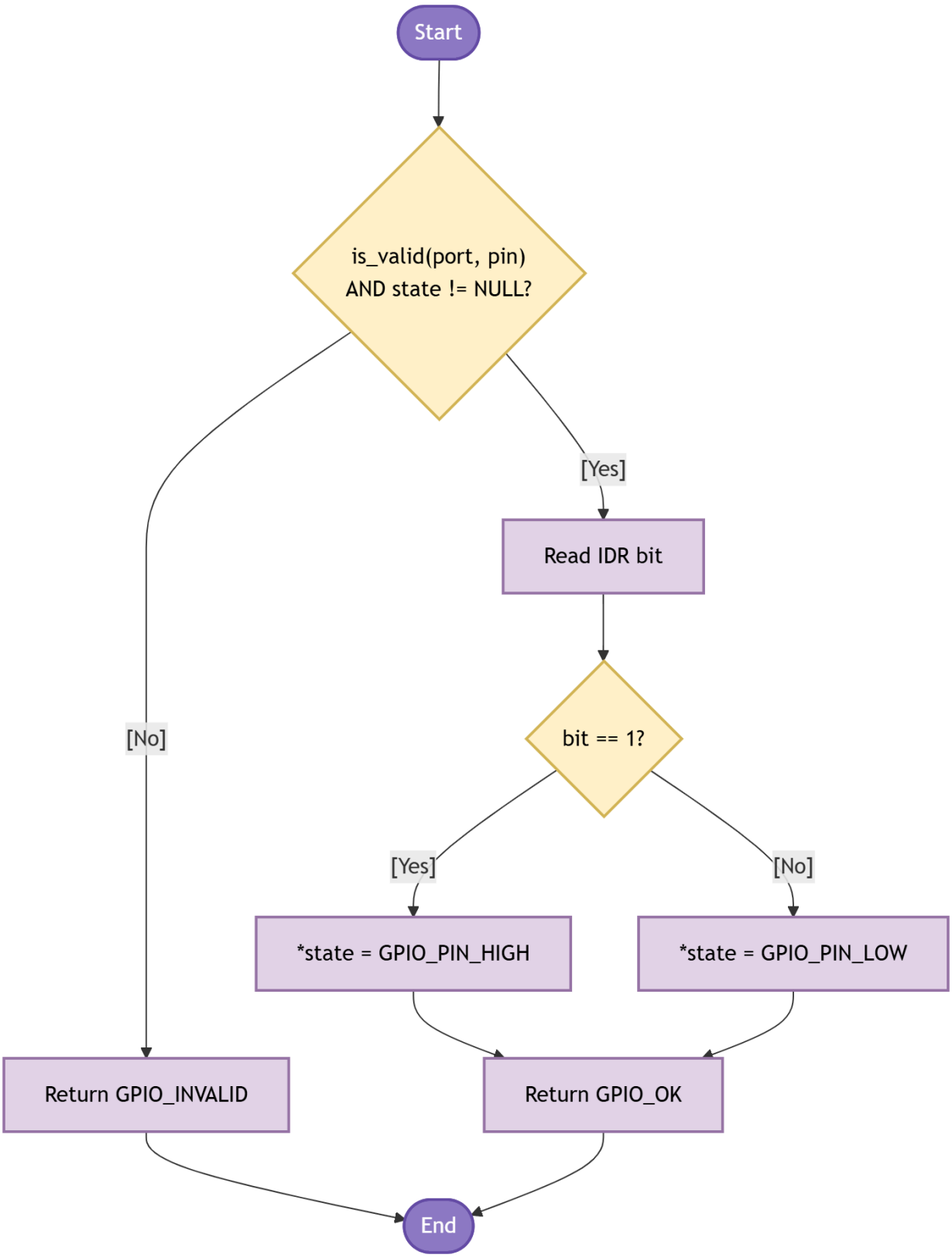

Figure 5. Activity Diagram - Driver Action with Conditional

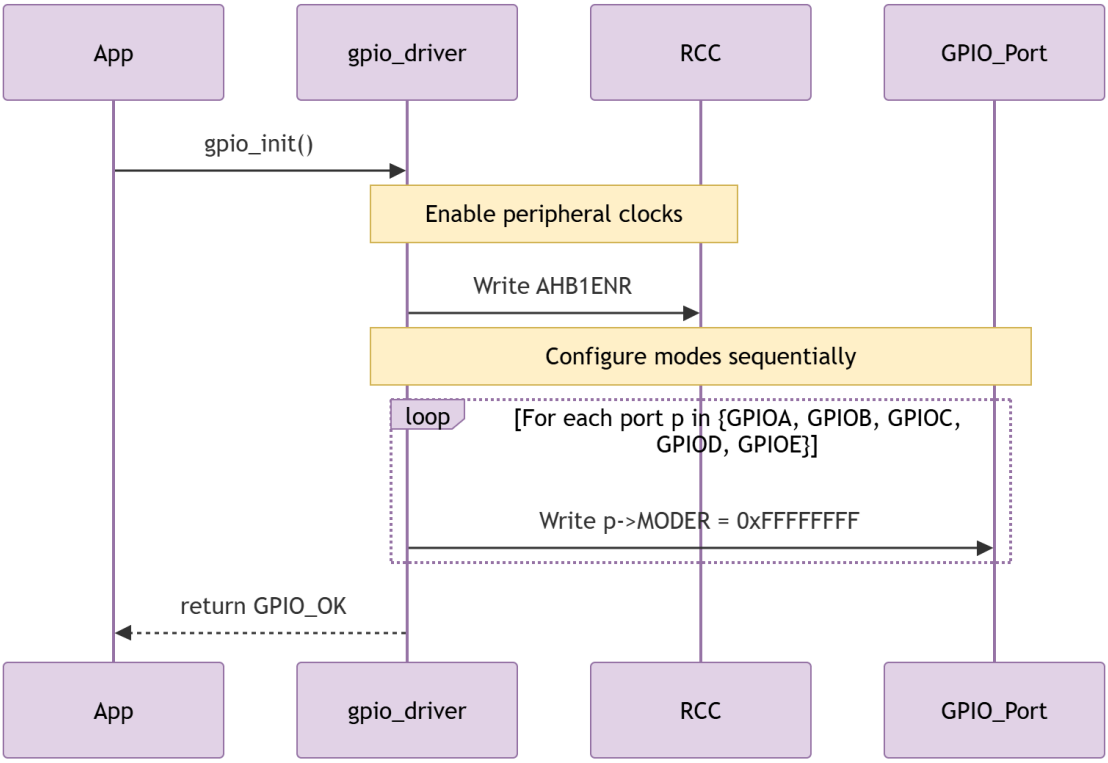

Figure 6. Sequence Diagram - Driver Action with Loop

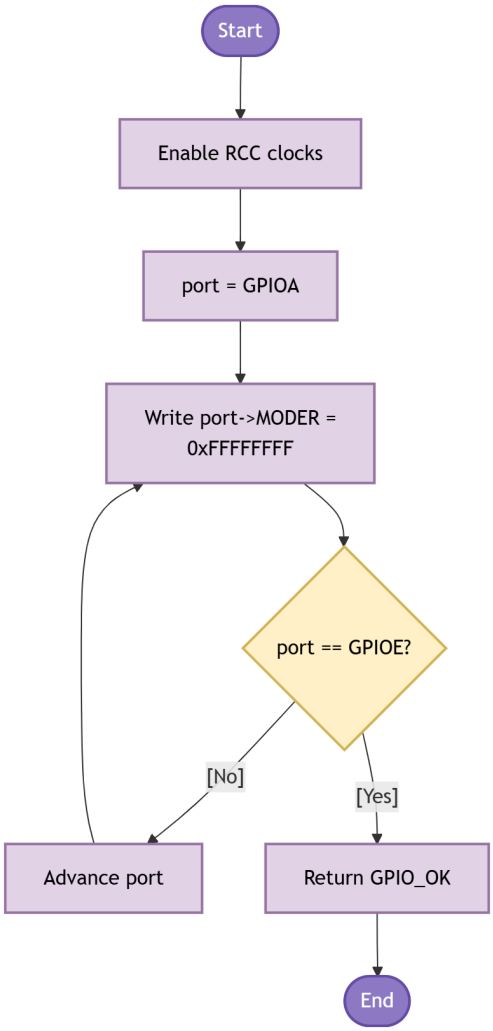

Figure 7. Activity Diagram - Driver Action with Loop

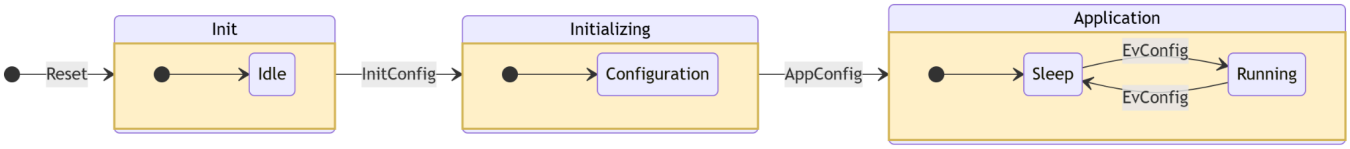

Figure 8. State Diagram - Driver

3.3. GPIO Driver

The GPIO driver works in two phases: initializes and pier-operation. The sections below walk through each function, explain what register it touches and why, and reference the diagram above.

Initialization

Call gpio_init() once at the start of main(), before any other GPIO function. It does two things in sequence (Figure 2, Figure 3):

- It writes RCC->AHB1ENR to enable the peripheral clocks for GPIOA through GPIOE in a single bus transaction. A port whose clock is off ignores all register writes, so this step must come first.
- It writes 0xFFFFFFFF to the MODER register of each port (Figure 6, Figure 7 show the loop). This value places all 16 pins of every port in analog mode -- the hardware reset state. Doing this explicitly guarantees a known starting configuration regardless of what the bootloader may have left behind.

If the application only needs one port -- for example, only GPIOA for the LED -- call gpio_initPort() instead. It enables the clock for that port alone using an if-else chain that matches the port pointer (GPIOA, GPIOB, ...) to the correct RCC bit. It does not touch MODER, so any pin configuration already on that port survives the call. Both led_init() and button_init() use gpio_initPort() internally.

Configuring a Pin Mode

After a port is clocked, call gpio_setPinMode() to set a specific pin. MODER has two bits per pin at positions [2*pin+1 : 2*pin]. The function:

- Clears those two bits: port->MODER &= ~(0x3U << (pin * 2))
- Writes the new mode: port->MODER |= (mode << (pin * 2))

Only those two bits change. The other 15 pins in the port are unaffected. gpio_setPinMode() does not check whether the port clock is on. If the clock is off, the write silently does nothing and GPIO_OK is returned anyway -- the caller is responsible for calling gpio_initPort() first.

Driving Output Pins (gpio_setPin and gpio_clearPin)

gpio_setPin() writes a 1 to bit [pin] of BSRR. gpio_clearPin() writes a 1 to bit [pin + 16] of BSRR. The hardware drives the pin to the requested state in the same write cycle.

This is the conditional operation in Figure 4 and Figure 5: depending on whether the goal is HIGH or LOW, a different half of the 32-bit BSRR word is targeted.

Because BSRR is write-only and a single write is all that is needed, there is no read-modify-write window. An interrupt that fires between a read and a write on ODR could corrupt the output; BSRR eliminates that risk entirely.

Toggling (gpio_togglePin)

gpio_togglePin() reads the current state of the pin from ODR, flips the bit with XOR, and writes it back: port->ODR ^= (1U << pin). This is a three-step read-modify-write. In the lab application the main loop is polling-only and no interrupt modifies GPIO ports, so the toggle is safe. If an ISR ever needs to toggle the same port, replace the XOR with a conditional set/clear using BSRR.

Reading an Input Pin (gpio_readPin)

gpio_readPin() samples IDR, which always reflects the actual pin voltage at the moment of the read, regardless of the pin's configured mode. Before touching IDR the function runs two checks (Figure 4, Figure 5):

- is_valid(port, pin): verifies port is not NULL and pin is in [0, 15].
- state != NULL: guards against a hard fault from dereferencing a null pointer.

If either check fails, the function returns GPIO_INVALID without touching any register. Debounce is entirely the caller's responsibility. IDR captures whatever voltage is on the pin at the instant of the read, so a bouncing contact gives inconsistent results for several milliseconds. The DEBOUNCE counter (#define DEBOUNCE 3) in main.c handles this for the user button.

Pin Mode State Machine (Figure 8)

A GPIO pin has four possible modes, shown in the state diagram. After gpio_init() resets MODER to 0xFFFFFFFF, every pin starts in ANALOG mode (bit pair = 0b11). gpio_setPinMode() is the only function that changes MODER. It can move a pin from any state to any other state in a single call. No other public function in this driver modifies MODER.

In the lab application: PA5 (LED) is set to OUTPUT. PC13 (button) is set to INPUT. No pin is set to ALT_FUNC because gpio_setAlternateFunction() was not implemented in this version.

4. Application Programming Interfaces

4.1. Data Types and Macros

gpio_status_t: Return type for all GPIO driver public functions. Every function validates its inputs and returns one of these codes before performing any register access.

gpio_mode_t: Selects the MODER encoding for a pin. The numeric values of the enumerators match the hardware 2-bit encoding directly, so no translation is needed inside the driver.

gpio_pin_t: Assigns symbolic names to pin numbers 0 through 15 (GPIO_PIN_0 … GPIO_PIN_15). Using this type prevents out-of-range pin values from being passed silently.

gpio_pin_state_t: Two-value type written by gpio_readPin to communicate the sampled logic level back to the caller (GPIO_PIN_LOW or GPIO_PIN_HIGH).

gpio_status_t: Enumerates the possible outcomes of a driver call.

Enumerator	Value	Meaning
GPIO_OK	0	The operation completed successfully; all parameters were valid and the register access was performed.
GPIO_ERROR	1	A generic error occurred. Reserved for hardware-level fault conditions not covered by GPIO_INVALID.
GPIO_INVALID	2	One or more input parameters failed validation: port is NULL or unrecognized, pin exceeds 15, or a required output pointer is NULL.

gpio_mode_t: Specifies the pin function written to the MODER register.

Enumerator	Value	MODER encoding / Use case
GPIO_MODE_INPUT	0x00	00 — configures the pin as a digital input; samples the external voltage level.

GPIO_MODE_OUTPUT	0x01	01 — configures the pin as a push-pull digital output; driven via BSRR or ODR.
GPIO_MODE_ALT_FUNC	0x02	10 — routes the pin to a hardware peripheral (UART, SPI, I2C, TIM, etc.) via AFR.
GPIO_MODE_ANALOG	0x03	11 — connects the pin to the ADC/DAC; digital buffers disabled. This is also the reset default set by gpio_init.

gpio_pin_state_t: Logic level returned by gpio_readPin.

Enumerator	Value	Meaning
GPIO_PIN_LOW	0	The sampled pin voltage corresponds to logic LOW (~0 V).
GPIO_PIN_HIGH	1	The sampled pin voltage corresponds to logic HIGH (~3.3 V).

4.2. Error Codes

All public GPIO driver functions return gpio_status_t. The three defined codes are listed below.

Code	Value	Definition
GPIO_OK	0	Returned when all parameter checks passed and the register operation completed without issue.
GPIO_ERROR	1	Reserved for generic hardware-level errors not attributable to a specific invalid parameter.
GPIO_INVALID	2	Returned when any input parameter is out of the valid range: port is NULL or does not match a known peripheral; pin > 15; alt_func > 15; or a required output pointer is NULL.

4.3. Function Definitions

gpio_init

Field	Name / Value	Description
Function Name	`gpio_init`	
Syntax	`gpio_status_t gpio_init(void)`	

Parameters (in)	None	—
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK upon completion. No parameter validation is performed so this function always returns GPIO_OK.
Description	Enables the AHB1 bus clocks for GPIO ports A through E by OR-writing all five enable bits into RCC→AHB1ENR in a single instruction. Following the clock enable, every port's MODER register is set to 0xFFFFFFFFU, placing all pins in analog mode — the STM32F4 lowest-power default and the safest starting configuration before individual pins are configured.	
Constraints	All five ports (A–E) are always initialized regardless of which ones the application will use. Must be invoked before any other GPIO driver function when full port coverage is required at startup.	
Available via	gpio_driver.h	

gpio_initPort

Field	Name / Value	Description
Function Name	`gpio_initPort`	
Syntax	`gpio_status_t gpio_initPort(GPIO_TypeDef *port)`	
Parameters (in)	port	GPIO_TypeDef* — CMSIS pointer to the target GPIO peripheral (GPIOA … GPIOE).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK if the clock was enabled. GPIO_INVALID if port is NULL or does not match any of the five supported GPIO base addresses.
Description	Enables the peripheral clock for a single GPIO port. The port pointer is first checked for NULL, then compared against the five known CMSIS port addresses (GPIOA … GPIOE) using an if-else chain. The matching RCC_AHB1ENR bit is set; all other port clocks are unaffected.	
Constraints	Pin modes are not modified; only the port clock is enabled. gpio_setPinMode must be called separately for each pin before it is used. The clock must remain enabled throughout the driver's operating lifetime.	

Available via	gpio_driver.h

gpio_setPinMode

Field	Name / Value	Description
Function Name	`gpio_setPinMode`	
Syntax	`gpio_status_t gpio_setPinMode(GPIO_TypeDef *port, gpio_pin_t pin, gpio_mode_t mode)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (in)	mode	gpio_mode_t — desired mode: GPIO_MODE_INPUT, GPIO_MODE_OUTPUT, GPIO_MODE_ALT_FUNC, or GPIO_MODE_ANALOG.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails (port NULL or pin > 15).
Description	Writes the requested mode to the pin's 2-bit MODER field at bit position (pin × 2). The field is first cleared with port->MODER &= ~(0x3U << (pin * 2U)), then set with port->MODER |= ((uint32_t)mode << (pin * 2U)). Parameter validation is performed by is_valid() before any register is touched.	
Constraints	The port clock must be enabled before calling. OTYPER, PUPDR, and AFR are not configured by this function; additional calls are required when non-default output type, pull resistors, or alternate function selection is needed.	
Available via	gpio_driver.h	

gpio_setPin

Field	Name / Value	Description
Function Name	`gpio_setPin`	

Syntax	`gpio_status_t gpio_setPin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails.
Description	Drives the specified pin to a HIGH logic level by writing (1U << pin) to the BSRR set half (bits 15:0). This is a single-write atomic operation — no prior read of ODR is needed and the output transitions to HIGH with no intermediate glitch state.	
Constraints	The pin must be configured as GPIO_MODE_OUTPUT before calling. Invoking on an input-configured pin has no visible effect on the pin voltage. Port clock must be enabled.	
Available via	gpio_driver.h	

gpio_clearPin

Field	Name / Value	Description
Function Name	`gpio_clearPin`	
Syntax	`gpio_status_t gpio_clearPin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails.
Description	Drives the specified pin to a LOW logic level by writing (1U << (pin + 16U)) to the BSRR reset half (bits 31:16). Like gpio_setPin, this is an atomic single-write operation that requires no prior read of ODR.	

Constraints	Same constraints as gpio_setPin: pin must be in output mode and port clock must be enabled.
Available via	gpio_driver.h

gpio_togglePin

Field	Name / Value	Description
Function Name	`gpio_togglePin`	
Syntax	`gpio_status_t gpio_togglePin(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails.
Description	Inverts the current output state of the specified pin by executing port->ODR ^= (1U << pin). The final pin state depends on the ODR value at the time of the call.	
Constraints	This is a non-atomic read-modify-write on ODR. If an ISR or another execution context can write to the same port's ODR concurrently, a race condition may corrupt the output state of other pins on that port. The caller must enforce mutual exclusion when this is a concern.	
Available via	gpio_driver.h	

gpio_readPin

Field	Name / Value	Description
Function Name	`gpio_readPin`	
Syntax	`gpio_status_t gpio_readPin(GPIO_TypeDef *port, gpio_pin_t pin, gpio_pin_state_t *state)`	

Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (inout)	None	—
Parameters (out)	state	gpio_pin_state_t* — pointer to a variable that will receive GPIO_PIN_LOW (0) or GPIO_PIN_HIGH (1).
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails or state is NULL.
Description	Reads the current logic level of the specified pin from IDR using port->IDR & (1U << pin). If the bit is set the function stores GPIO_PIN_HIGH at *state; otherwise it stores GPIO_PIN_LOW. Both the is_valid() check and the NULL guard on state are applied before any register is accessed. The function works correctly on both input and output configured pins since IDR always reflects the actual pin voltage.	
Constraints	state must not be NULL. Port clock must be enabled. GPIO_INVALID is returned for any failed check without further distinguishing which specific argument was invalid.	
Available via	gpio_driver.h	

gpio_setAlternateFunction

Field	Name / Value	Description
Function Name	`gpio_setAlternateFunction`	
Syntax	`gpio_status_t gpio_setAlternateFunction(GPIO_TypeDef *port, gpio_pin_t pin, uint8_t alt_func)`	
Parameters (in)	port	GPIO_TypeDef* — pointer to the target GPIO peripheral.
Parameters (in)	pin	gpio_pin_t — target pin number (GPIO_PIN_0 … GPIO_PIN_15).
Parameters (in)	alt_func	uint8_t — alternate function number (0–15) as defined in the STM32F411RE datasheet for the target pin.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	gpio_status_t	GPIO_OK on success. GPIO_INVALID if is_valid() fails or alt_func > 15.

Description	Configures the alternate function mapping for a pin by writing to the AFR registers (GPIO_TypeDef.AFR[0] for pins 0–7, AFR[1] for pins 8–15). The 4-bit field for the target pin is first cleared, then the alt_func value is written in. This must be called in addition to gpio_setPinMode(GPIO_MODE_ALT_FUNC) to fully configure a pin for peripheral use.
Constraints	The pin mode must be set to GPIO_MODE_ALT_FUNC separately using gpio_setPinMode. The port clock must be enabled. Valid alternate function numbers and their peripheral mappings are device-specific; refer to the STM32F411xC/E datasheet alternate function table.
Available via	gpio_driver.h

4.4. Internal Function Definitions

is_valid

Field	Name / Value	Description
Function Name	`is_valid`	
Syntax	`static int is_valid(GPIO_TypeDef *port, gpio_pin_t pin)`	
Parameters (in)	port	GPIO_TypeDef* — port pointer to validate.
Parameters (in)	pin	gpio_pin_t — pin number to validate.
Parameters (inout)	None	—
Parameters (out)	None	—
Return value	int	1 if both arguments are valid; 0 if port is NULL or pin exceeds 15.
Description	Private helper that centralizes the two most common parameter checks across all public API functions: a NULL check on the port pointer and a range check on the pin number. By consolidating these guards here, each public function avoids duplicating the same validation logic.	
Constraints	Defined as static in gpio_driver.c; not visible or callable from outside the driver. Does not validate the mode or alt_func arguments — those are checked inline in their respective functions.	

5. Future Enhancements

The following improvements may be incorporated in upcoming revisions of the GPIO driver:

- **Output type configuration (OTYPER):** Add a function to select push-pull or open-drain output mode per pin, required for I2C bus lines and certain LED driver topologies.
- **Pull-up / pull-down configuration (PUPDR):** Introduce a function to program internal pull-up or pull-down resistors per pin via the PUPDR register, removing dependency on external board pull resistors.
- **Output speed control (OSPEEDR):** Provide a function to set the slew rate of each output pin (low, medium, high, very high) for signal integrity management at higher switching frequencies.
- **Port F and G support:** Extend gpio_initPort to recognize GPIOF and GPIOG for higher pin-count STM32F4 variants such as the STM32F407.
- **External interrupt support (EXTI):** Add configuration functions for GPIO-sourced interrupts, including edge selection (rising, falling, both) and NVIC priority setup, enabling event-driven designs without polling.
- **Port-level batch read / write:** Provide gpio_readPort and gpio_writePort to operate on all 16 pins of a port in a single IDR or ODR access, useful for parallel bus interfaces.
- **Configuration locking (LCKR):** Expose a gpio_lockPin function to permanently prevent reconfiguration of critical pins after initialization using the LCKR register.
- **Atomic toggle via BSRR:** Replace the ODR XOR in gpio_togglePin with a BSRR-based approach to make the operation interrupt-safe without requiring external critical sections.
- **Thread-safety wrappers:** Add PRIMASK-based critical section macros around non-atomic ODR operations to make the driver safe in RTOS or multi-interrupt contexts.